

208. 实现 Trie (前缀树)

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium	(Not Available)	(Not Available)	-	-	0

- **Tags:** 设计, 字典树, 哈希表, 字符串
- **Companies:** (Not Available)

Trie (发音类似 “try”) 或者说 前缀树是一种树形数据结构，用于高效地存储和检索字符串数据集中的键。这一数据结构有相当多的应用情景，例如自动补完和拼写检查。

请你实现 Trie 类：

- `Trie()` 初始化前缀树对象。
- `void insert(String word)` 向前缀树中插入字符串 `word` 。
- `boolean search(String word)` 如果字符串 `word` 在前缀树中，返回 `true`（即，在检索之前已经插入）；否则，返回 `false` 。
- `boolean startsWith(String prefix)` 如果之前已经插入的字符串 `word` 的前缀之一为 `prefix`，返回 `true`；否则，返回 `false` 。

示例：

输入

```
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]  
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
```

输出

```
[null, null, true, false, true, null, true]
```

解释

```
Trie trie = new Trie();  
trie.insert("apple");  
trie.search("apple");    // 返回 True  
trie.search("app");      // 返回 False  
trie.startsWith("app");  // 返回 True  
trie.insert("app");  
trie.search("app");      // 返回 True
```

提示：

- `1 <= word.length, prefix.length <= 2000`
- `word` 和 `prefix` 仅由小写英文字母组成
- `insert`、`search` 和 `startsWith` 调用次数 总计不超过 $3 * 10^4$ 次

Discussion | Solution

解决方法

1. 问题理解

目标：实现一个 Trie (前缀树) 数据结构，支持插入字符串、查找完整字符串、查找字符串前缀三个操作。**Trie** 特点：- 根节点不代表任何字符。- 每个节点代表一个字符。- 从根节点到某个节点的路径构成一个字符串（通常是前缀）。- 每个节点可能有多个子节点，对应不同的字符。- 通常在节点中标记该节点是否是一个完整单词的结尾。

2. 思路分析

Trie 节点设计 每个 Trie 节点需要包含以下信息：

1. 子节点指针：指向其子节点的引用。由于题目说明只有小写英文字母，我们可以使用一个固定大小的数组（大小为 26）或者一个哈希表来存储子节点。
 - 数组实现： `children = [None] * 26`。 `children[0]` 对应 'a'， `children[1]` 对应 'b'，以此类推。优点是查找快 ($O(1)$)，空间固定。
 - 哈希表实现： `children = {}`。 `children['a']` 对应 'a' 的子节点。优点是空间更灵活（只存储存在的字符），但查找理论上是 $O(1)$ 平均，最坏 $O(\Sigma)$ ， Σ 是字符集大小。对于固定的小写字母，数组更常用。
2. 结束标记：一个布尔值 `is_end_of_word` (或类似名称)，标记从根节点到当前节点的路径是否构成一个完整的、已插入的单词。

操作实现

1. **Trie()** 初始化：
 - 创建一个根节点 `root`。根节点本身不代表字符，其 `children` 指向第一层字符节点。
2. **insert(word)** 插入字符串：
 - 从根节点 `root` 开始，遍历 `word` 中的每个字符 `char`。
 - 对于当前节点 `node` 和字符 `char`：
 - 计算字符 `char` 对应的索引（例如 `index = ord(char) - ord('a')`）。
 - 检查 `node.children[index]` 是否存在。
 - 如果不存在，创建一个新的 Trie 节点，并将其放入 `node.children[index]`。
 - 将当前节点移动到子节点： `node = node.children[index]`。
 - 遍历完 `word` 的所有字符后，到达最后一个字符对应的节点。将该节点的 `is_end_of_word` 标记设置为 `True`。
3. **search(word)** 查找完整字符串：
 - 从根节点 `root` 开始，遍历 `word` 中的每个字符 `char`。
 - 对于当前节点 `node` 和字符 `char`：
 - 计算字符 `char` 对应的索引 `index`。
 - 检查 `node.children[index]` 是否存在。
 - 如果不存在，说明 `word` 不在前缀树中，直接返回 `False`。

- 如果存在,将当前节点移动到子节点:`node = node.children[index]`。
- 遍历完 `word` 的所有字符后, 到达最后一个字符对应的节点 `node`。
- 检查该节点的 `is_end_of_word` 标记。如果为 `True`, 说明 `word` 确实被插入过, 返回 `True`; 否则返回 `False` (可能只是一个前缀, 但不是一个完整插入的词)。

4. `startsWith(prefix)` 查找前缀:

- 与 `search` 操作非常类似, 只是最后一步不同。
- 从根节点 `root` 开始, 遍历 `prefix` 中的每个字符 `char`。
- 对于当前节点 `node` 和字符 `char`:
 - 计算字符 `char` 对应的索引 `index`。
 - 检查 `node.children[index]` 是否存在。
 - 如果不存在, 说明 `prefix` 不是任何已插入单词的前缀, 直接返回 `False`。
 - 如果存在, 将当前节点移动到子节点:`node = node.children[index]`。
- 遍历完 `prefix` 的所有字符后, 成功到达了 `prefix` 最后一个字符对应的节点。这意味着存在以 `prefix` 开头的单词, 直接返回 `True` (不需要检查 `is_end_of_word`)。

3. 代码实现 (Python)

```
class TrieNode:
    """Trie 树的节点定义"""
    def __init__(self):
        # 使用大小为 26 的数组存储子节点, 索引 0-25 对应 'a'-'z'
        self.children = [None] * 26
        # 标记从根到当前节点是否形成一个完整的单词
        self.is_end_of_word = False

class Trie:
    """Trie 树实现"""
    def __init__(self):
        """
        初始化 Trie 树, 创建一个空的根节点。
        """
        self.root = TrieNode() # 根节点不代表任何字符

    def insert(self, word: str) -> None:
        """
        向前缀树中插入一个单词。
        Args:
            word: 要插入的字符串。
        """
        node = self.root # 从根节点开始
        for char in word:
            index = ord(char) - ord('a') # 计算字符对应的索引
            # 如果该字符对应的子节点不存在, 则创建新节点
```

```

        if node.children[index] is None:
            node.children[index] = TrieNode()
        # 移动到子节点
        node = node.children[index]
    # 单词遍历结束后, 将当前节点的结束标记设为 True
    node.is_end_of_word = True

def search(self, word: str) -> bool:
    """
    在前缀树中查找一个完整的单词是否存在。
    Args:
        word: 要查找的字符串。
    Returns:
        如果单词存在于 Trie 中, 则返回 True, 否则返回 False。
    """
    node = self.root # 从根节点开始
    for char in word:
        index = ord(char) - ord('a') # 计算字符对应的索引
        # 如果路径中断 (字符对应的子节点不存在), 则单词不存在
        if node.children[index] is None:
            return False
        # 移动到子节点
        node = node.children[index]
    # 单词遍历结束后, 必须检查当前节点是否是一个单词的结尾
    return node.is_end_of_word

def startsWith(self, prefix: str) -> bool:
    """
    检查前缀树中是否存在以指定前缀开头的单词。
    Args:
        prefix: 要查找的前缀字符串。
    Returns:
        如果存在以 prefix 开头的单词, 则返回 True, 否则返回 False。
    """
    node = self.root # 从根节点开始
    for char in prefix:
        index = ord(char) - ord('a') # 计算字符对应的索引
        # 如果路径中断 (字符对应的子节点不存在), 则此前缀不存在
        if node.children[index] is None:
            return False
        # 移动到子节点
        node = node.children[index]
    # 如果成功遍历完前缀的所有字符, 说明此前缀存在
    return True

```

```
# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)
```

4. 复杂度分析

设 N 为操作次数, L 为单词/前缀的最大长度, Σ 为字符集大小 (这里是 26)。

- **insert(word):**
 - 时间复杂度: $O(L)$ 。需要遍历单词的每个字符, 并在每个字符处进行常数时间的操作 (数组索引或哈希表查找/插入)。
 - 空间复杂度: $O(L)$ 。在最坏情况下 (插入的单词没有公共前缀), 每个字符都需要创建一个新的 Trie 节点。
- **search(word):**
 - 时间复杂度: $O(L)$ 。需要遍历单词的每个字符。
 - 空间复杂度: $O(1)$ 。仅需要常数额外空间。
- **startsWith(prefix):**
 - 时间复杂度: $O(L)$ 。需要遍历前缀的每个字符。
 - 空间复杂度: $O(1)$ 。仅需要常数额外空间。

Trie 树整体空间复杂度: $O(N * L * \Sigma)$ 是一个非常宽松的上界。实际空间复杂度取决于插入单词的公共前缀数量。如果公共前缀很多, 空间会远小于上界, 最坏情况是 $O(N * L)$ (所有单词无公共前缀)。更精确地说是 $O(\text{所有节点数} * \Sigma)$, 或者 $O(\text{所有节点数} + \text{所有边数})$ 。节点数最多为 $N * L$ 。

5. 如何在面试中讲解

1. 解释 **Trie** (前缀树) 的概念:
 - “Trie 是一种树形结构, 专门用于高效存储和检索字符串。它的关键思想是利用字符串的公共前缀来节省空间和提高查询效率。”
 - “每个节点代表路径上的一个字符, 从根到一个节点的路径构成一个前缀。节点通常需要一个标记来指示它是否是一个完整单词的结尾。”
2. 节点设计:
 - “每个 Trie 节点需要存储两部分信息:”
 - “指向子节点的指针: 因为我们处理的是小写字母, 可以用一个大小为 26 的数组 `children`, `children[i]` 指向代表第 i 个字母的子节点。”
 - “结束标记 `is_end_of_word`: 一个布尔值, 表示从根到此节点的路径是否构成一个已插入的完整单词。”
 - “根节点是空的, 不代表任何字符。”
3. **insert** 操作:
 - “从根节点开始, 沿着 `word` 的字符路径向下走。”
 - “对于每个字符, 计算其在 `children` 数组中的索引。”
 - “如果对应的子节点不存在, 就创建一个新的 `TrieNode`。”
 - “然后移动到该子节点。”

- “当 word 遍历完毕，将最后一个节点的 `is_end_of_word` 设为 `True`。”
4. `search` 操作:
- “同样从根节点开始，沿着 word 的字符路径向下走。”
 - “如果在任何一步发现对应的子节点不存在，说明 word 不在树中，返回 `False`。”
 - “如果成功走完了 word 的所有字符，到达最后一个节点，必须检查该节点的 `is_end_of_word` 是否为 `True`。只有为 `True` 才表示这个完整的单词被插入过。”
5. `startsWith` 操作:
- “与 `search` 非常类似，也是从根节点沿着 prefix 的字符路径向下走。”
 - “如果在任何一步发现对应的子节点不存在，说明此前缀不存在，返回 `False`。”
 - “如果成功走完了 prefix 的所有字符，不需要检查 `is_end_of_word`，直接返回 `True`，因为只要路径存在就说明有单词以此为前缀。”
6. 复杂度分析:
- “假设单词/前缀最大长度为 L 。”
 - “插入、查找、前缀查找的时间复杂度都是 $O(L)$ 。”
 - “空间复杂度：插入操作最坏 $O(L)$ 新增空间。整个 Trie 的空间取决于所有插入单词的总字符数以及它们的重叠程度。”
7. 应用场景:
- “自动补全、拼写检查、IP 路由（最长前缀匹配）等。”